



Com Sys Notes Arlz-Copy

Computer System (Swinburne University of Technology (Vietnam))



messages.pdf_cover_qr_code_label

Converting Hex into decimals

A-F = 10 to 15

Example: given number 7CF

C = 12

F = 15

Multiply each digit with the powers of 16 (16^x) starting from units place of the number

$$7CF = (7 \cdot 16^2) + (12 \cdot 16^1) + (15 \cdot 16^0)$$

$$= (7 \cdot 256) + (12 \cdot 16) + (15 \cdot 1)$$

$$= 1792 + 192 + 15$$

$$= 1999$$

Logic gates

NOT (the inverter): accepts an input and outputs the opposite

AND: all inputs must be positive for output to be positive (1/0N)

NAND: same as AND but the output is inverse (NOT gate at output)

OR: at least one input must be positive for output to be positive (both inputs can be positive)

NOR: same as OR but the output is inverse (NOT gate at output)

XOR (exclusive or): only one input can be positive for a positive output. If both are positive, negative output given

XNOR: all inputs must be same (both 1 or 0) for a positive output otherwise output is negative

Why do we clock gates?

- Allows us control over the speed of the processor
- Electrical path length (delay) may be different for different inputs
- Needed for converting serial to parallel and back

How to clock gates?

- AND and/or NAND the clock pulse with the gate input
- Set a control bit to an AND gate input

Flip flops

- An electrical circuit with two stable states used to store 1 bit (binary) of data

Flip flops and latches (aka unclocked flip flops)

- Flip flops are edge triggered, meaning they change their output when a specific condition on the input is met (clock pulses)
- Latches are level triggered, changing their output whenever there is change in input

RS flip flops (reset – set flip flops)

Initially, the state output can be in either SET or RESET state

$S = 1$ and $R = 0$, output = 1 (on)

$S = 0$ and $R = 1$, output = 0 (off)

$S = 0$ and $R = 0$, holds current state (output will remain same as previous)

$S = 1$ and $R = 1$, undefined/forbidden state (error)

A clocked RS flip flop will need two additional AND gates

RS flip flops can also be latches (unclocked flip flops)

D flip flops (data/delay flip flop)

- Has two inputs which control the flip flop, data and clock input. When clock input is high, the data is transferred to output. When clock input is low, the output retains its previous state (ignores data input).

(when clock is high and Q = output)

- $D = 0$, $Q = 0$
- $D = 1$, $Q = 1$

T flip flops (toggle flip flop)

- Needs a clock pulse to operate
- Can be constructed using RS, D and JK flip flops

- Called toggle flip flop because the bit stored will be flipped, e.g., 1 to 0, 0 to 1
- Whenever T is 1, the next state output (Q_{t+1}) will be complemented of the current state output (Q), making it toggle
- Whenever T is 0, next state output (Q_{t+1}) is the same as the current state input (Q)

JK flip flops

- Has two inputs, J(set) and K(reset) and their corresponding outputs are Q and Q'
- Can be synchronous or asynchronous
- In synchronous mode, the state will be changed with the clock signal(pulses)
- In asynchronous mode, the change is independent from the clock
- If input J(set) and CLK are both high, the clock will be in the set state
- If input K(reset) and CLK are both high, the clock will be in the reset state
- If both inputs and CLK are high, it will be in the toggle state
- If the JK flip flop's CLK isn't high, the outputs will retain its previous states

- Counters:

A device which stores (sometimes displays) the number of times a particular event/process has occurred. It will also increment the instruction pointer.

Asynchronous counter (serial, ripple counter): No universal clock, first flip flop is driven by main clock and the clock input for the rest of the other flip flops are driven by the output of the previous flip flop.

For ripple counters, responding to clock's falling edge: count up

Responding to clock's rising edge: count down

Feeding Q' to the display (led) will also count down

A base-level counter means it starts with LSB and moves towards MSB (example the number 6 will be shown as 011 instead of 110)

Synchronous counter (parallel): Has one global clock, connected to each CLK input of the flip flops within the counter. It does not have cumulative delay.

- Shift registers:

A group of flip flops with a common clock used to store multiple bits of data and move the data from one flip flop to another. It can shift left or right. Shift registers can also double and halve binary values (Serial IN Parallel OUT, Parallel IN Serial OUT, respectively)

(There can be an overlap between counters and shift registers, it all really depends on the specific requirements of the application)

--> Registers are storage locations within the CPU, used to store and quickly access data. They can also have specific uses like holding the program counter, stack pointer or other special values.

- Memory units:

Stacks (hardware and software): they offer a way to organize and store memory with random access. Addresses within a stack can be accessed if they are on 'top' or are pointed at by the stack pointer. Stacks allows a process (code) to be interrupted and supports recursion (when a function is called, a new frame is added to the call stack to store information about the function call like local variables and the return address)

Hardware stacks are created out of dedicated shift registers

Example: a 4-bit deep stack will require a bi-directional shift register made up of 4 D ffs and 4 controlled gates

A 3-bit wide and 5-bit deep stack will require 3 shift registers made of 5 D ffs

Software stacks are typically defined in RAM and are limited to the amount of addressable RAM

RAM (Random access memory)

DDR (Double data rate): technique in computing where the computer bus transfers data on both falling/rising edges of a clock cycle

Most types of RAMs are volatile, meaning they lose stored data when power is cut off. However, Flash memory (used in USB) and FRAM are non-volatile

To access a specific bit within RAM, you need the bit number, byte and word address, page/block/bank (these 3 are used to describe organizational structures within memory hierarchy)

Polling is a technique where the program/device actively checks the status of a condition/resource at regular intervals instead of relying on immediate notifications/interrupts

An 'atomic' interrupt is an interrupt that cannot be interrupted

An encoder combines a set of electrical states to form an n-bit number

A decoder reverses what an encoder does, converting the n-bit number into a set of separate binary values and is also a way of controlling/selecting part of an electrical circuit

Parallel cables/devices are more susceptible to electrical interference

Even parity means the number of 1s (including the parity bit) is even, example: 1100101 -> 11001010 (even ratio of 0-1s). It is an error checking method and what it does above is to ensure that the number of 1s is either even or 0; it's commonly used to detect single bit errors

Fixed-point numbers are representations of real numbers with a fixed amount of numbers to the right of the decimal point.

Sign extension is where the bit representation of a signed number is extended whilst preserving its sign. It copies the sign bit (MSB) into additional bits to maintain the same value when the number is represented using a larger number of bits.

BCD (binary coded decimal): each decimal digit is represented by its 4-bit binary equivalent. Example: 0111 0001 is equal to 71. BCD can also be represented by a fixed-point number

The role of the linker when compiling programming language into machine instructions is to find implementations of unresolved symbols in the program being compiled

Von Neumann architecture is a computer design that uses a single memory space to store both program instructions and data.

Harvard architecture uses separate memory spaces for program instructions and data. Doing this, Harvard's architecture allows simultaneous access to both, enabling faster instruction fetches and data access.

ARM assembly

Program counter (PC) keeps track of the next instruction to execute

Link register (LR) is a special used register used to store the return address of a subroutine call (function call)

LDR instruction copies memory at given address into a register

LSL (logical shift left) is used to double values (in binary) and for every number assigned to it, it will double.

Example: this puts 1+8 (think binary instead of decimal) and doubles it twice

```
mov r0, #1
orr r0, #8
lsl r0, #2
```

1 in binary is 0001 and 8 is 1000. Because of the orr instruction, the value in R0 is now 1001. Using the LSL, we will now double/move the value by 2, which gives us 100100 (equivalent to 36)

In the instruction LDR r0, [r1, #8], #8 is an offset and r1 now contains a memory address (if it hasn't already). Offset is the immediate value added to the base address to calculate the memory address.

Indirect addressing refers to a method where a memory address which is not directly specified in the instruction but stored in a register.

Indexed addressing involves specifying a memory address using a combination of a base register and an index register. The address is calculated by adding together the content of both the base register and index register.

In ARM, ABI (application binary interface) defines conventions on how functions, data structures, and system calls should be organized and accessed in a binary executable.

